66.7
Pass

Pooja Biradar

Other

PDF generated at: 23 Feb 2026 13:30:37 UTC

View this report on HackerRank

Score

66.7% • 150.0 / 225

scored in eClerx - FO Java, Unix & Python Assessment in 77 min 47 sec on 23 Feb 2026 17:39:55 IST

Candidate Information

Email	sbpooja942@gmail.com
Test	eClerx - FO Java, Unix & Python Assessment
Candidate Packet	View
Taken on	23 Feb 2026 17:39:55 IST
Time taken	77 min 47 sec/ 90 min
Work Experience	4 years
Invited by	Vipul
Invited on	23 Feb 2026 17:09:40 IST

Skill Distribution

No.	Skill	Score
1	Bash Intermediate	100%

2	Java Intermediate	0%
3	Python Intermediate	100%

Tags Distribution

Bash	100%	Medium	66%
Shell Scripting	100%	Arrays	100%
Strings	100%	Java	0%
Exceptions	0%	OOP	50%
Context Manager	100%	Python	100%
Classes	100%		

Questions

Linux/Unix • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
1	Bash: Pattern Matching Coding	Bash (Intermediate)	75/75	-	7 min 47 sec	✓

Java (Intermediate) • 0 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
2	Java: Vending Machine Coding	Java (Intermediate)	0/75	-	1 hour 4 min 17 sec	⊗

Python (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
3	DB Context Coding	Python (Intermediate)	75/75	-	4 min 58 sec	✓

1. Bash: Pattern Matching

Correct

Coding Bash Medium Shell Scripting Arrays Strings

Question description

Given an array of strings, count the strings that contain at least one uppercase character and output the result to STDOUT.

Example:
my_array = ["FirstWord", "Word2", "thirdword"]

The match_uppercase function will return 2 since "FirstWord" and "Word2" each contain an uppercase letter.

▼ INPUT FORMAT FOR CUSTOM TESTING

The only line contains the elements of *my_array*.

▼ SAMPLE CASE 0

Sample Input For Custom Testing

```
Abc
bcd
Efg
def
cDe
```

Sample Output

```
3
```

Explanation

There are 3 strings containing at least one uppercase character: "Abc", "Efg", and "cDe".

▼ SAMPLE CASE 1

Sample Input For Custom Testing

```
SingleWord
```

Sample Output

```
1
```

Explanation

There is 1 string containing at least one uppercase character: "SingleWord".

Candidate's Solution

Language used: **BASH**

```
#!/usr/bin/env bash
readarray -t my_array </dev/stdin

# Input array is read and stored in to my_array variable.
# You can view the code by pressing > button above.

function match_uppercase() {
  typeset -a data=("$@")
  # Write your code here
  count=0

  for str in "${data[@]}";do
       case "$str" in
           *[A-Z]*)
```

```
                count=$((count+1))
                ;;
        esac
  done
  echo "$count"
}

match_uppercase "${my_array[@]}"
exit 0
```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
TestCase 0	Easy	Sample	Success	1	0.0025 sec	1.63 KB
TestCase 1	Easy	Sample	Success	1	0.0027 sec	1.75 KB
TestCase 2	Easy	Hidden	Success	1	0.0038 sec	1.75 KB
TestCase 3	Easy	Hidden	Success	8	0.0043 sec	1.88 KB
TestCase 4	Easy	Hidden	Success	8	0.0049 sec	1.75 KB
TestCase 5	Easy	Hidden	Success	8	0.0039 sec	1.75 KB
TestCase 6	Easy	Hidden	Success	8	0.0247 sec	2.63 KB
TestCase 7	Easy	Hidden	Success	8	0.0254 sec	2.63 KB
TestCase 8	Easy	Hidden	Success	8	0.0263 sec	2.63 KB
TestCase 9	Easy	Hidden	Success	8	0.0247 sec	2.63 KB

TestCase 10	Easy	Hidden	Success	8	0.0243 sec	2.63 KB
TestCase 11	Easy	Hidden	Success	8	0.0257 sec	2.63 KB

No comments.

2. Java: Vending Machine

Incorrect

Coding Medium Java Exceptions OOP

Question description

Design a vending machine that serves 3 drinks: "Coke", "Fanta", and "Sprite".

Create a *VendingMachine* class that implements the following methods:

- *void registerDrink(int buttonNumber, Drink drink)*: Registers a type of drink with a particular *buttonNumber* of the vending machine. Here, *Drink* is an interface provided in the locked stub.
- *ServedDrinkSummary(int buttonNumber, int money)*: Returns the Drink that is served when *buttonNumber* is pressed, and an amount of money is entered by a customer.

The class contains two attributes:

- *change*: the units of currency to return to the consumer
- *drink*: the drink requested

There are three potential error conditions:

- If the machine has run out of the drink, an *OutOfStockException* should be thrown with the message "{drinkName} is out of stock".
- If the money the user enters is insufficient to buy the drink, an *InsufficientMoneyException* should be thrown.
- If the drink is out of stock and the user has not entered enough money, throw an *OutOfStockException* with the previously mentioned message.

The *OutOfStockException, InsufficientMoneyException*, and *ServedDrinkSummary* classes are provided in the locked stub.

The locked stub validates the *VendingMachine* class by processing some requests.

Constraints

- $1 \leq totalNumberOfRequests \leq 10^4$
- $0 \leq buttonNumber \leq 2$
- $0 \leq money \leq 100$
- $0 \leq quantity$ available of each $drink \leq 10^4$
- $0 \leq price$ per unit of each $drink \leq 10^4$
- No two drinks are assigned the same *buttonNumber.*

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains three integers that represent the *quantity* of *Coke*, *Fanta,* and *Sprite* in the vending machine respectively.
The second line contains three integers that represent the *price* per unit of *Coke*, *Fanta,* and *Sprite* respectively.
The third line contains three integers that represent the *buttonNumbers* assigned to *Coke*, *Fanta,* and *Sprite* respectively.
The fourth line contains an integer, *totalNumberOfRequests .*
The next *totalNumberOfRequests* lines contain the *buttonNumber* pressed and *money* inserted separated by a space.

▼ SAMPLE CASE 0

Sample Input For Custom Testing

```
STDIN    Function
-----    --------
2 1 0    quantity Coke = 2, Fanta = 1, Sprite = 0
5 10 15  price Coke = 5, Fanta = 10, Sprite = 15
0 1 2    buttonNumbers Coke = 0, Fanta = 1, Sprite = 2
4        totalNumberOfRequests = 4
0 20     buttonNumber = 0, money = 20 for the first request
2 20     buttonNumber = 2, money = 20 for the second request
2 5      ...
1 5
```

Sample Output

```
Vending machine set up
Coke 15
Sprite is out of stock
Sprite is out of stock
Insufficient money
```

Explanation

- *0 20:* A request to process a can of *Coke* and *20* units of money is provided to the vending machine. *Coke* is in stock and costs *5* units of money so *15* units in change is returned to the consumer. The locked stub prints "*Coke 15".*
- *2 20:* A request to process a can of *Sprite* and *20* units of money is provided to the vending machine. *Sprite* is out of stock so an *OutOfStockException* is thrown, and the stub prints "*Sprite is out of stock".*
- *2 5:* A request to process a can of *Sprite* and *5* units of money is provided to the vending machine. *Sprite* is out of stock and the amount of *money* is insufficient so an *OutOfStockException* is thrown and the stub prints, "*Sprite is out of stock".*
- *1 5:* A request to process a can of *Fanta* and *5* units of money is provided. Since the price of a unit of *Fanta is 10* and it is in stock, an *InsufficientMoneyException* is thrown, and the stub prints "*Insufficient money".*

Interviewer guidelines

▼ SOLUTION

Solution for Java 7, 8 and 15

```
class VendingMachine{
    HashMap<Integer, Drink> drinkRelatedToButton = new HashMap<>();

    void registerDrink(int buttonIdx, Drink drink){
        drinkRelatedToButton.put(buttonIdx, drink);
    }

    ServeDrinkSummary dispatch(int buttonPressed, int money)throws OutOfStockException, Insuffi
cientMoneyException{
        ServeDrinkSummary serveDrinkSummary = new ServeDrinkSummary();
        serveDrinkSummary.servedDrink = drinkRelatedToButton.get(buttonPressed);
        serveDrinkSummary.change = money - serveDrinkSummary.servedDrink.getPrice();
```

```
        if(serveDrinkSummary.servedDrink.getQuantityLeft() == 0){
            throw new OutOfStockException(serveDrinkSummary.servedDrink.getName() + " is out of st
ock");
        }

        if(serveDrinkSummary.change < 0){
            throw new InsufficientMoneyException();
        }
        return serveDrinkSummary;
    }
}
```

Candidate's Solution

Language used: **Java 8**

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

interface Drink{
    int getPrice();
    String getName();
    int getQuantityLeft();
    void serveDrink();
}

class ServeDrinkSummary{
    Drink servedDrink;
    int change;
}

class OutOfStockException extends Exception{
    public OutOfStockException(String message){
        super(message);
    }
}

class InsufficientMoneyException extends Exception{
    public InsufficientMoneyException(){
        super();
    }
}
import java.io.*;
import java.util.Map;
```

```
import java.util.HashMap;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

interface Drink{
    int getPrice();
    String getName();
    int getQuantityLeft();
    void serveDrink();
}

public class ServeDrinkSummary{
    private Drink drink;
    private int change;
}




class OutOfStockException extends Exception{
    public OutOfStockException(String message){
        super(message);
    }
}

class InsufficientMoneyException extends Exception{
    public InsufficientMoneyException(){
        super();
    }
}

class VendingMachine{

    private Map<Integer, Drink> drinkMap = new HashMap<>();
    private Map<Integer, Integer> quantityMap = new HashMap<>();

    void registerDrink(int buttonIdx, Drink drink){
        drinkMap.put(buttonIdx, drink);
        quantityMap.put(buttonIdx, 1);
    }


    ServeDrinkSummary dispatch(int buttonPressed, int money)
```

```
        throws OutOfStockException, InsufficientMoneyException{

    Drink drink = drinkMap.get(buttonPressed);
    if(drink == null){
        throw new OutOfStockException("Drink not available");
    }
    int quantity = quantityMap.get(buttonPressed);
    if(quantity <=0 ){
        throw new OutOfStockException(drink.getName() + " is out of stock");
    }
    if(money<drink.getPrice()){
        throw new InsufficientMoneyException();
    }
    quantityMap.put(buttonPressed, quantity - 1);

    int change=money-drink.getPrice();
    drink.serveDrink();



    return new ServeDrinkSummary(drink, change);
    }
}

class Solution{

    public static void main(String[] args) throws IOException{
        String arr[];
        int quantity[] = new int[3], pricePerUnit[] = new int[3], buttonAssigned[] = new int[3];

        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        PrintWriter out = new PrintWriter(System.out);

        arr = br.readLine().split(" ");
        for(int i = 0; i < 3; i++){
            quantity[i] = Integer.parseInt(arr[i]);
        }

        arr = br.readLine().split(" ");
        for(int i = 0; i < 3; i++){
            pricePerUnit[i] = Integer.parseInt(arr[i]);
        }

        arr = br.readLine().split(" ");
```

```
        for(int i = 0; i < 3; i++){
            buttonAssigned[i] = Integer.parseInt(arr[i]);
        }

        VendingMachine vendingMachine = new VendingMachine();
        Drink coke = new Coke(pricePerUnit[0], "Coke", quantity[0]);
        Drink fanta = new Fanta(pricePerUnit[1], "Fanta", quantity[1]);
        Drink sprite = new Sprite(pricePerUnit[2], "Sprite", quantity[2]);
        vendingMachine.registerDrink(buttonAssigned[0], coke);
        vendingMachine.registerDrink(buttonAssigned[1], fanta);
        vendingMachine.registerDrink(buttonAssigned[2], sprite);
        out.println("Vending machine set up");

        int totalNumberOfRequests = Integer.parseInt(br.readLine().trim());
        while(totalNumberOfRequests-- > 0){
            arr = br.readLine().split(" ");
            int buttonPressed = Integer.parseInt(arr[0]),
                money = Integer.parseInt(arr[1]);
            try{
                ServeDrinkSummary serveDrinkSummary = vendingMachine.dispatch(buttonPressed, money);
                serveDrinkSummary.servedDrink.serveDrink();
                out.println(serveDrinkSummary.servedDrink.getName() + " " + serveDrinkSummary.change);
            }catch(OutOfStockException e){
                out.println(e.getMessage());
            }catch(InsufficientMoneyException e){
                out.println("Insufficient money");
            }
        }

        out.flush();
        out.close();
    }
}

class Coke implements Drink{
    private int price;
    private String name;
    private int quantityLeft;

    public Coke(int price, String name, int quantityLeft){
        this.price = price;
        this.name = name;
        this.quantityLeft = quantityLeft;
    }
```

```java

    public int getPrice(){
        return this.price;
    }

    public String getName(){
        return this.name;
    }

    public int getQuantityLeft(){
        return this.quantityLeft;
    }


    public void serveDrink(){
        if(this.quantityLeft > 0)
            this.quantityLeft -= 1;
    }
}

class Fanta implements Drink{
    private int price;
    private String name;
    private int quantityLeft;

    public Fanta(int price, String name, int quantityLeft){
        this.price = price;
        this.name = name;
        this.quantityLeft = quantityLeft;
    }

    public int getPrice(){
        return this.price;
    }

    public String getName(){
        return this.name;
    }

    public int getQuantityLeft(){
        return this.quantityLeft;
    }

    public void serveDrink(){
        if(this.quantityLeft > 0)
            this.quantityLeft -= 1;
```

```
    }
}

class Sprite implements Drink{
    private int price;
    private String name;
    private int quantityLeft;

    public Sprite(int price, String name, int quantityLeft){
        this.price = price;
        this.name = name;
        this.quantityLeft = quantityLeft;
    }

    public int getPrice(){
        return this.price;
    }

    public String getName(){
        return this.name;
    }

    public int getQuantityLeft(){
        return this.quantityLeft;
    }

    public void serveDrink(){
        if(this.quantityLeft > 0)
            this.quantityLeft -= 1;
    }
}
class Solution{

    public static void main(String[] args) throws IOException{
        String arr[];
        int quantity[] = new int[3], pricePerUnit[] = new int[3], buttonAssigned[] = new int[3];

        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        PrintWriter out = new PrintWriter(System.out);

        arr = br.readLine().split(" ");
        for(int i = 0; i < 3; i++){
            quantity[i] = Integer.parseInt(arr[i]);
        }
```

```java

        arr = br.readLine().split(" ");
        for(int i = 0; i < 3; i++){
            pricePerUnit[i] = Integer.parseInt(arr[i]);
        }

        arr = br.readLine().split(" ");
        for(int i = 0; i < 3; i++){
            buttonAssigned[i] = Integer.parseInt(arr[i]);
        }

        VendingMachine vendingMachine = new VendingMachine();
        Drink coke = new Coke(pricePerUnit[0], "Coke", quantity[0]);
        Drink fanta = new Fanta(pricePerUnit[1], "Fanta", quantity[1]);
        Drink sprite = new Sprite(pricePerUnit[2], "Sprite", quantity[2]);
        vendingMachine.registerDrink(buttonAssigned[0], coke);
        vendingMachine.registerDrink(buttonAssigned[1], fanta);
        vendingMachine.registerDrink(buttonAssigned[2], sprite);
        out.println("Vending machine set up");

        int totalNumberOfRequests = Integer.parseInt(br.readLine().trim());
        while(totalNumberOfRequests-- > 0){
            arr = br.readLine().split(" ");
            int buttonPressed = Integer.parseInt(arr[0]),
                money = Integer.parseInt(arr[1]);
            try{
                ServeDrinkSummary serveDrinkSummary = vendingMachine.dispatch(buttonPressed, money);
                serveDrinkSummary.servedDrink.serveDrink();
                out.println(serveDrinkSummary.servedDrink.getName() + " " + serveDrinkSummary.change);
            }catch(OutOfStockException e){
                out.println(e.getMessage());
            }catch(InsufficientMoneyException e){
                out.println("Insufficient money");
            }
        }

        out.flush();
        out.close();
    }
}

class Coke implements Drink{
    private int price;
    private String name;
```

```
    private int quantityLeft;

    public Coke(int price, String name, int quantityLeft){
        this.price = price;
        this.name = name;
        this.quantityLeft = quantityLeft;
    }

    public int getPrice(){
        return this.price;
    }

    public String getName(){
        return this.name;
    }

    public int getQuantityLeft(){
        return this.quantityLeft;
    }

    public void serveDrink(){
        if(this.quantityLeft > 0)
            this.quantityLeft -= 1;
    }
}

class Fanta implements Drink{
    private int price;
    private String name;
    private int quantityLeft;

    public Fanta(int price, String name, int quantityLeft){
        this.price = price;
        this.name = name;
        this.quantityLeft = quantityLeft;
    }

    public int getPrice(){
        return this.price;
    }

    public String getName(){
        return this.name;
    }

    public int getQuantityLeft(){
```

```
        return this.quantityLeft;
    }

    public void serveDrink(){
        if(this.quantityLeft > 0)
            this.quantityLeft -= 1;
    }
}

class Sprite implements Drink{
    private int price;
    private String name;
    private int quantityLeft;

    public Sprite(int price, String name, int quantityLeft){
        this.price = price;
        this.name = name;
        this.quantityLeft = quantityLeft;
    }

    public int getPrice(){
        return this.price;
    }

    public String getName(){
        return this.name;
    }

    public int getQuantityLeft(){
        return this.quantityLeft;
    }

    public void serveDrink(){
        if(this.quantityLeft > 0)
            this.quantityLeft -= 1;
    }
}
```

Result: Compilation Failed

Compile Message

```
ServeDrinkSummary.java:69: error: constructor ServeDrinkSummary in class
ServeDrinkSummary cannot be applied to given types;
  return new ServeDrinkSummary(drink, change);
      ^
```

```
  required: no arguments
  found: Drink,int
  reason: actual and formal argument lists differ in length
ServeDrinkSummary.java:113: error: cannot find symbol
            serveDrinkSummary.servedDrink.serveDrink();
                             ^
  symbol:   variable servedDrink
  location: variable serveDrinkSummary of type ServeDrinkSummary
ServeDrinkSummary.java:114: error: cannot find symbol
            out.println(serveDrinkSummary.servedDrink.getName() + " " + serveDrinkSummary.change);
                                        ^
  symbol:   variable servedDrink
  location: variable serveDrinkSummary of type ServeDrinkSummary
ServeDrinkSummary.java:114: error: change has private access in ServeDrinkSummary
            out.println(serveDrinkSummary.servedDrink.getName() + " " + serveDrinkSummary.change);
                                                                        ^
4 errors
```

No comments.

3. DB Context

Correct

Coding | Medium | Context Manager | Python | Classes | OOP

Question description

Implement a class called *database_context_manager* that acts as a context manager for database connections.

The context manager should:

- Create a database connection when entering the context by calling the *_init_db()* method (already defined in the *Database* class)
- Close the connection when exiting the context by calling the *_close_db()* method (also defined in the *Database* class)

Given a string *username* consisting of lowercase English letters, define and complete the *database_context_manager class*.

For a successful simulation:

- The input string username must be passed to *_init_db()* as an argument.
- The connection string returned from *_init_db()* must be passed to *_close_db()* as an argument to close the connection.

Example

Given input *username* = "Sam", if the connection string returned by *_init_db()* is "abc":

- "Sam" must be passed as an argument to *_init_db()*
- "abc" must be passed to *_close_db()*

Function Description

Complete the class *database_context_manager* in the editor with the following parameter(s):

string username: the string to pass to *_init_db()*

Constraints

- $1 \leq |\ username\ | \leq 10^5$
- *username* consists of lowercase English letters.

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line and the only line of the test case contains a string, *username*.

▼ SAMPLE CASE 0

Sample Input For Custom Testing

```
Robin
```

Sample Output

```
Username matched, database connection initialized
Database running...
Connection string matched, database connection closed
```

Explanation

Given, *username = "Robin",* the steps processed are:

- Implement the class *database_context_manager.*
- Call the *_init_db()* method with the correct parameter. Output received is "Username matched, database connection initialized".
- The output received is "Database running..." to signal that the database is currently in a running state.

- Call the *_close_db()* method with the correct parameter. The output received is "Connection string matched, database connection closed".

▼ SAMPLE CASE 1

Sample Input For Custom Testing

```
Sam
```

Sample Output

```
Username matched, database connection initialized
Database running...
Connection string matched, database connection closed
```

Explanation

Given, *username = "Sam"*, the steps processed are:

- Implement the class *database_context_manager.*
- Call the *_init_db()* method with the correct parameter. Output received is "Username matched, database connection initialized".
- The output received is "Database running..." to signal that the database is currently in a running state.
- Call the *_close_db()* method with the correct parameter. The output received is "Connection string matched, database connection closed".

Interviewer guidelines

▼ SOLUTION

```
class database_context_manager():

    def __init__(self, username):
        self.username = username
        self.connectionString = ""
        self.obj = Database()

    def __enter__(self):
        self.connectionString = self.obj._init_db(self.username)

    def __exit__(self, exc_type, exc_value, exc_traceback):
```

```
    self.obj._close_db(self.connectionString)
```

Candidate's Solution

Language used: **Python 3**

```python
#!/bin/python

import math
import os
import random
import re
import sys

fptr = open(os.environ['OUTPUT_PATH'], 'w')

class Database:

    totalCount: int = 0
    count1: int = 0
    count2: int = 0
    inputString: int = ""
    __connString: int = ""

    def _init_db(self, username: str) -> str:

        Database.totalCount += 1
        Database.__connString = ""

        if username == Database.inputString:

            Database.count1 += 1

            le: int = random.randint(1, 100000)
            for x in range(le):
                nxt: int = random.randint(1, 100000)
                nxt %= 26
                Database.__connString += chr(97 + nxt)

            fptr.write('Username matched, database connection initialized\n')

        return Database.__connString

    def _close_db(self, connectionString: str) -> None:

        Database.totalCount += 1
```

```
        if Database.__connString == connectionString:
            Database.count2 += 1
            fptr.write('Connection string matched, database connection closed\n')


#
# Complete the class 'database_context_manager' below.
#
# The class accepts STRING username as parameter.
#

class database_context_manager:

    def __init__(self, username):
        self.username = username
        self.connection = None
    def __enter__(self):
        self.db = Database()
        self.connection = self.db._init_db(self.username)
        return self
    def __exit__(self,exc_type,exc_value,traceback):
        self.db._close_db(self.connection)

    # write your code here
if __name__ == '__main__':

    username: str = input()
    Database.inputString = username

    with database_context_manager(username) as dbSimulator:
        fptr.write('Database running...\n')

    tc: int = Database.totalCount
    cnt1: int = Database.count1
    cnt2: int = Database.count2

    if cnt1 != 1 or cnt2 != 1 or tc != 2:
        fptr.write('Functions not correctly called\n')
    fptr.close()

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED

TestCase 0	Easy	Sample	Success	1	0.0616 sec	10.8 KB
TestCase 1	Easy	Sample	Success	1	0.0462 sec	10.8 KB
TestCase 2	Easy	Sample	Success	1	0.0696 sec	10.9 KB
TestCase 3	Easy	Hidden	Success	5	0.0347 sec	10.8 KB
TestCase 4	Easy	Hidden	Success	5	0.2361 sec	10.9 KB
TestCase 5	Easy	Hidden	Success	6	0.0292 sec	10.9 KB
TestCase 6	Easy	Hidden	Success	6	0.1114 sec	10.9 KB
TestCase 7	Easy	Hidden	Success	10	0.0692 sec	10.9 KB
TestCase 8	Easy	Hidden	Success	10	0.0442 sec	10.6 KB
TestCase 9	Easy	Hidden	Success	10	0.0259 sec	10.8 KB
TestCase 10	Easy	Hidden	Success	10	0.0747 sec	11 KB
TestCase 11	Easy	Hidden	Success	10	0.0692 sec	10.9 KB

No comments.